

Strategic Threat Analysis Lab

Encrypted Traffic Analysis and Contextual Risk Evaluation in a Controlled Environment

Prepared by Grace Luhanga

Ubuntu, Wireshark, TLS Analysis, and Stellar Cyber

Project at a glance: Captured and analyzed HTTP and TLS traffic, established baseline browser behavior, identified contextual outlier traffic to we.tl, and investigated full-session metadata to determine whether observed activity represented normal encrypted communication or potential data transfer risk.

Executive Summary

I designed and implemented a focused threat analysis lab using Ubuntu and Wireshark to investigate how legitimate-looking encrypted traffic can still require security review. The project centered on building a defender's workflow for distinguishing baseline browser activity from contextually sensitive outbound connections. After establishing normal HTTP activity such as Firefox connectivity checks and expected browser traffic, I identified outbound TLS communication to we.tl, a legitimate file-sharing service that can also be abused for unauthorized data transfer. I then performed session-level analysis by tracing the associated destination IP address, reviewing the full TCP and TLS lifecycle, and evaluating whether the traffic exhibited behaviors consistent with elevated risk. This project strengthened my ability to solve security problems by moving from visibility to interpretation: not simply detecting traffic, but determining whether it matters.

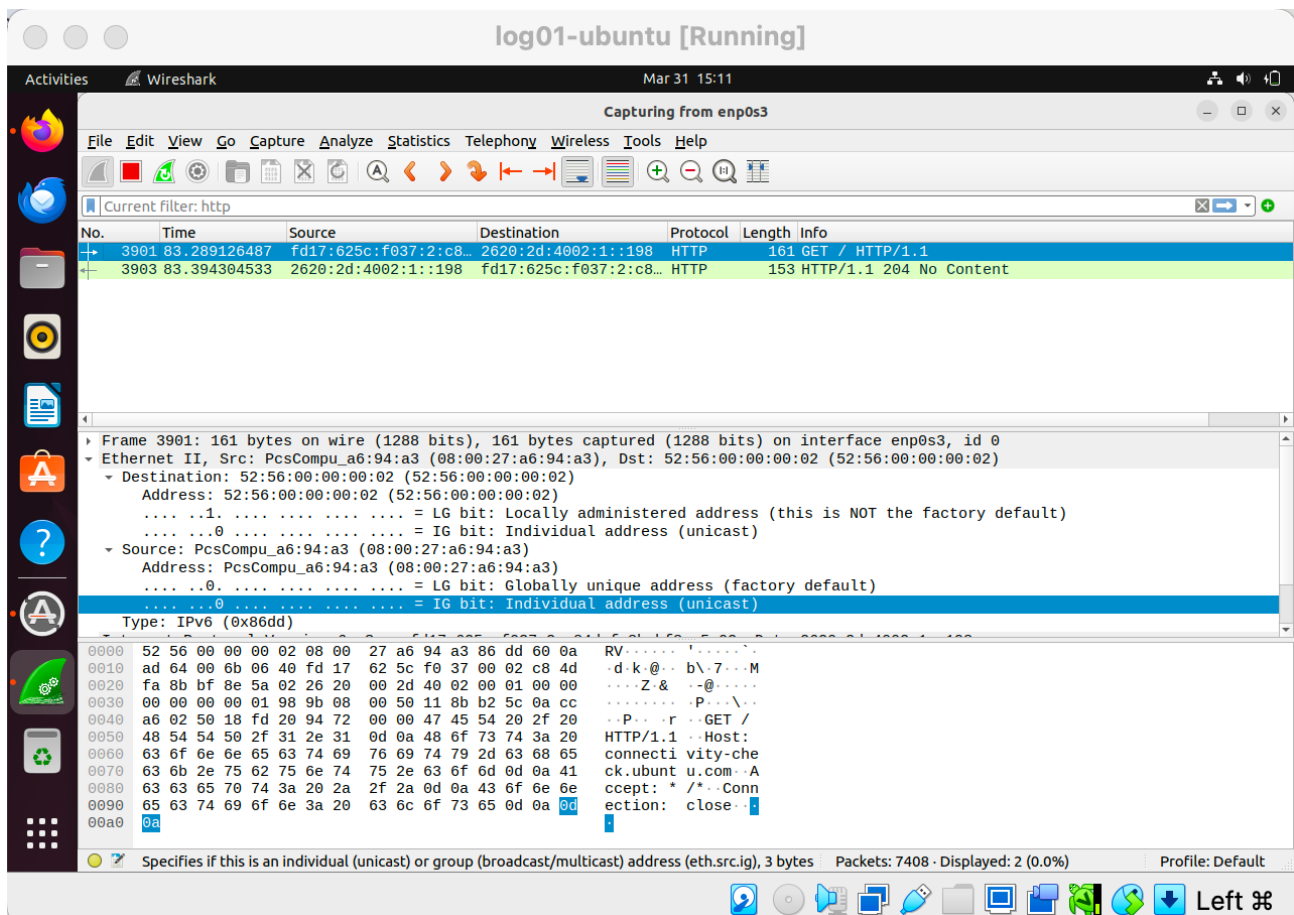


Figure 1. Wireshark data collection setup on the Ubuntu NAT-facing interface.

Project Objective

The goal of this project was to investigate encrypted outbound traffic from a defender's perspective and demonstrate the ability to move from packet capture into risk-based interpretation. Rather than stopping at traffic identification, I wanted to show that I could establish a clean baseline, isolate an external service, follow the full connection lifecycle, and explain why a technically normal session can still be operationally important.

Scenario

This project was structured as a practical analyst simulation. The scenario assumed that a security analyst needed to review browser-based network activity, identify whether an external file-sharing service appeared in traffic, and determine whether that activity was benign, suspicious, or worthy of additional monitoring. The problem being solved was not simply “what traffic exists,” but “what traffic deserves attention.”

Environment and Architecture

The analysis environment consisted of a monitored Ubuntu virtual machine running in Oracle VirtualBox with Wireshark installed for packet capture. Traffic was captured on the NAT-facing interface (enp0s3) to observe real outbound browser communication. The workflow focused on three stages: baseline HTTP analysis, TLS-based domain detection using Server Name Indication (SNI), and IP-based session investigation to understand frequency, duration, and connection behavior.

Component	Role	Key Configuration	Observed Data
Ubuntu VM	Monitored system / packet capture	VirtualBox VM; Wireshark on enp0s3	10.0.2.15
Wireshark	Traffic analysis tool	HTTP, TLS, and IP-based filtering	Packet-level visibility
Browser traffic	Observed activity source	Firefox connectivity checks and external browsing	Baseline + contextual outlier traffic

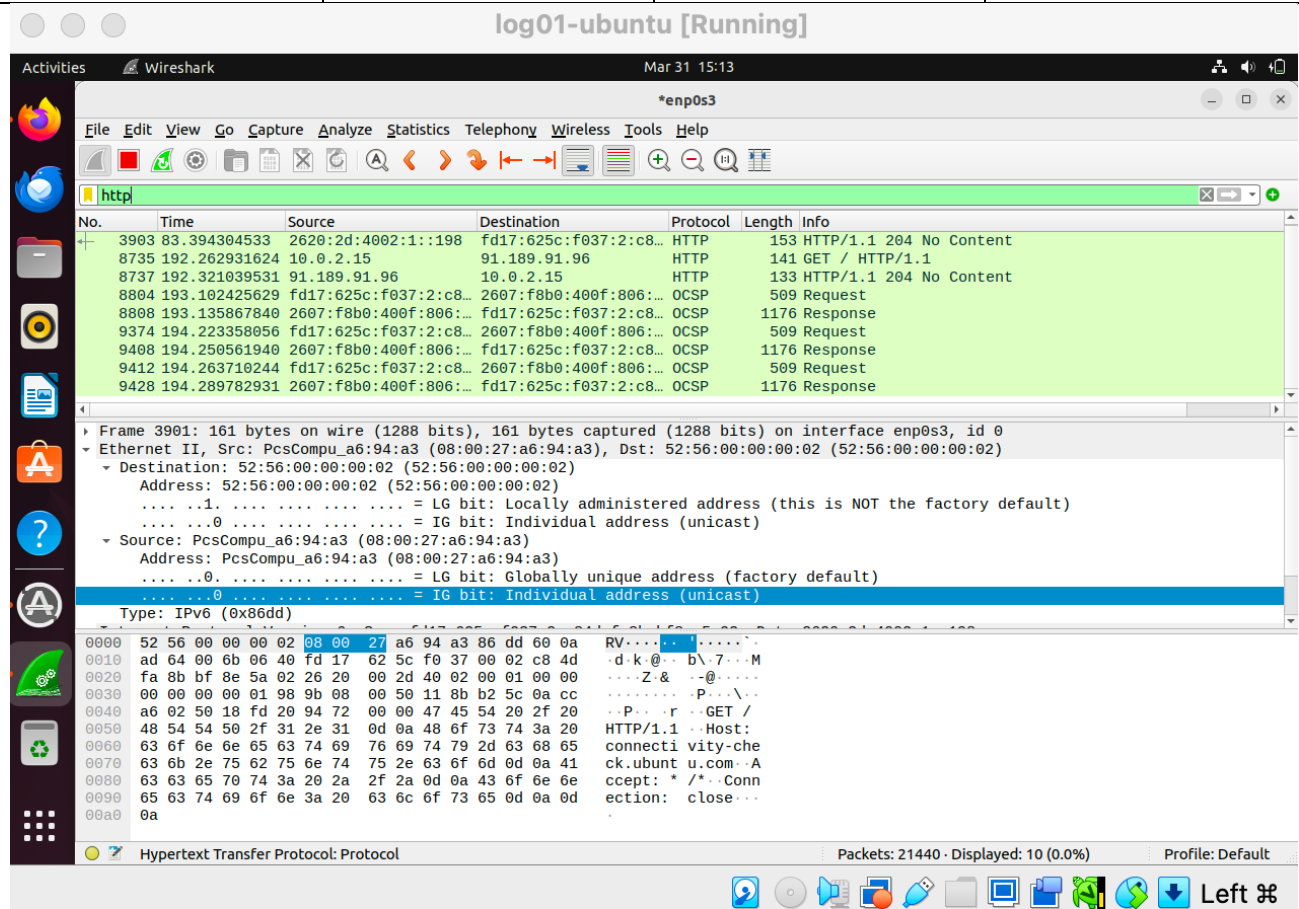


Figure 2. Technical control of analysis showing targeted HTTP traffic inspection in Wireshark.

Methodology

1. Captured live browser traffic on Ubuntu using Wireshark on the NAT-facing interface (enp0s3).
2. Applied an http filter to establish baseline browser and system-generated behavior.
3. Identified expected traffic such as connectivity checks and normal web requests.
4. Applied a TLS SNI filter to detect communication with we.tl.
5. Extracted the associated destination IP address and filtered the session using ip.addr == 3.169.202.120.
6. Reviewed handshake, encrypted application data, keep-alive behavior, and clean session termination.
7. Interpreted the observed activity in the context of modern data exfiltration risks and legitimate web-service abuse.

Key Activities and Findings

1. Baseline Browser Activity and Validation

I began by validating that Wireshark was capturing meaningful outbound browser traffic on enp0s3. Using an http filter, I observed expected requests associated with Firefox connectivity checks and routine browsing behavior. This baseline was important because it gave me a working reference point for what normal looked like before introducing a contextually riskier destination.

Security relevance: Establishing a normal baseline is a problem-solving step in security operations because analysts must know what is expected before they can recognize what is unusual.

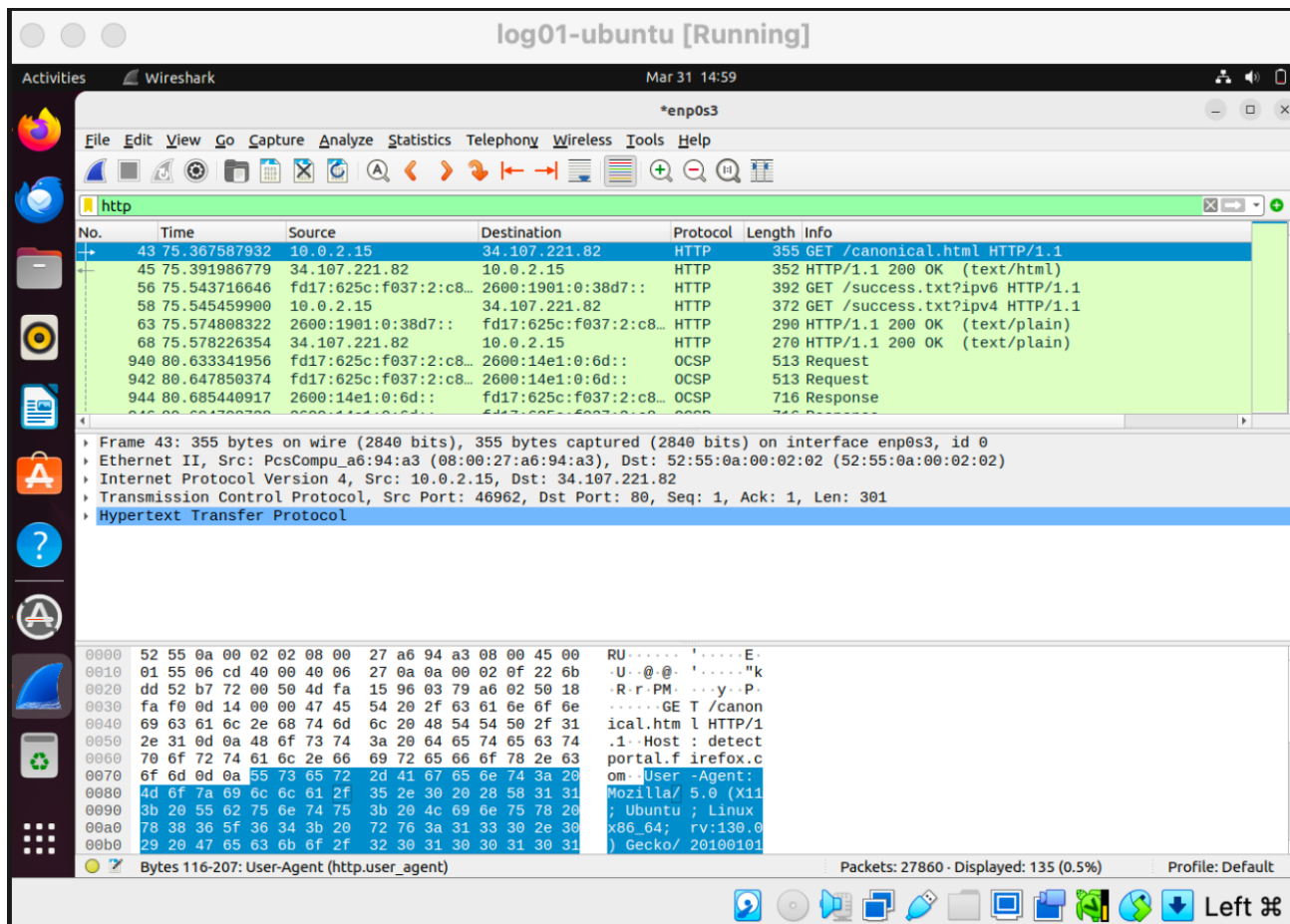


Figure 3. Baseline HTTP activity showing expected browser communication and connectivity-check traffic.

2. Detection of Contextual Outlier Traffic

After confirming baseline behavior, I shifted from general traffic inspection to targeted detection. Because modern web activity is frequently encrypted, I used TLS Server Name Indication (SNI) to identify communication with we.tl. This allowed me to confirm that the host initiated encrypted connections to an external file-sharing service even though packet contents could not be read.

Security relevance: This step demonstrates how analysts solve visibility limitations in encrypted environments by using metadata, such as domains revealed during TLS negotiation, to detect potentially important external communication.

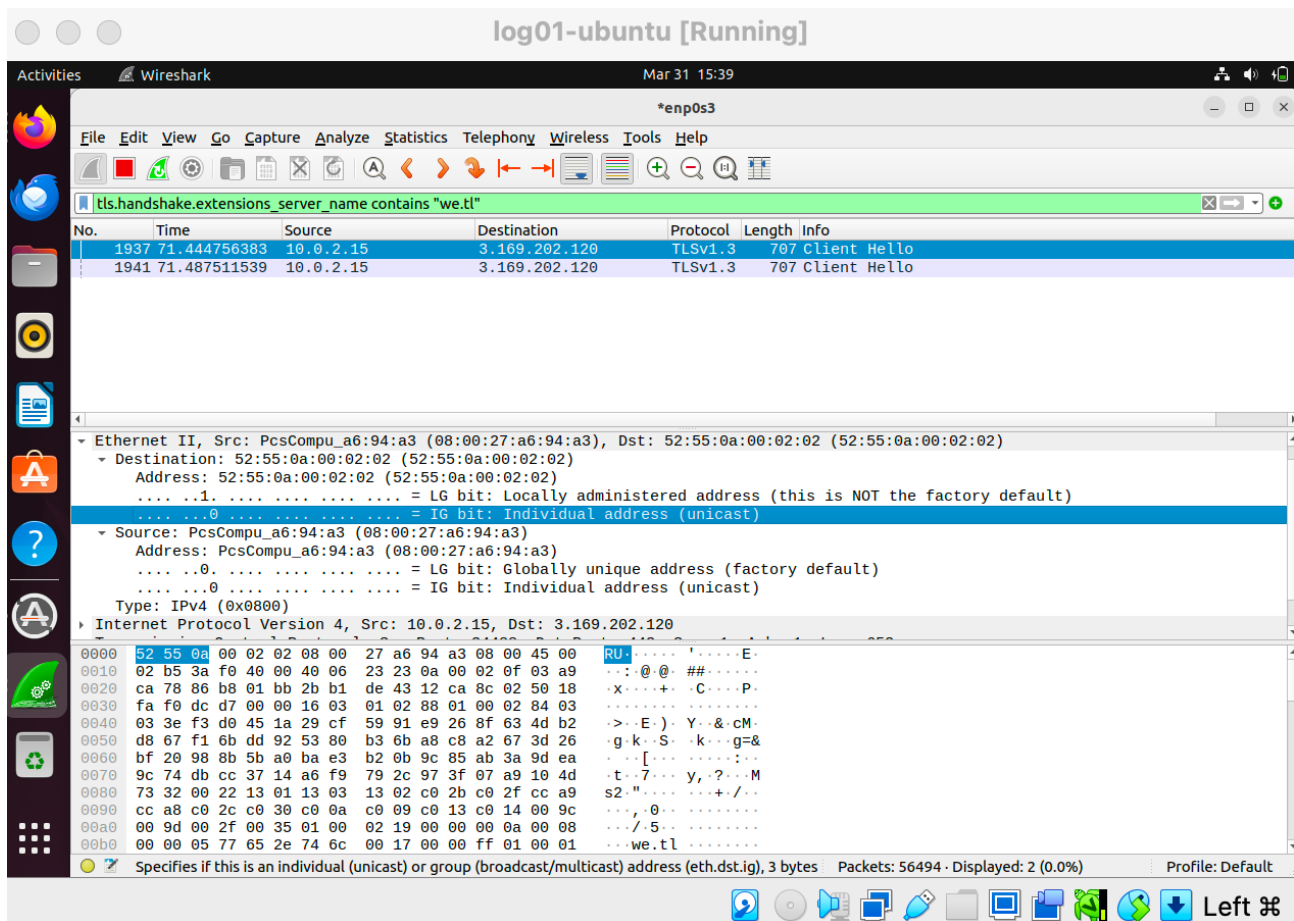


Figure 4. TLS handshake evidence confirming outbound communication to we.tl.

3. Session-Level Investigation and Depth of Analysis

To move beyond simple detection, I extracted the destination IP address associated with we.tl and filtered the traffic using `ip.addr == 3.169.202.120`. This allowed me to investigate the entire session rather than a single packet. I observed the TCP handshake, TLSv1.3 Client Hello, encrypted application data, repeated keep-alive behavior, and clean session termination. The session pattern was technically normal, but the destination remained contextually sensitive because file-sharing services can be used for unauthorized transfer of data.

Security relevance: This activity shows the difference between identifying traffic and solving the underlying security question. A technically valid encrypted session may still require attention depending on where it goes, how long it persists, and what business context surrounds it.

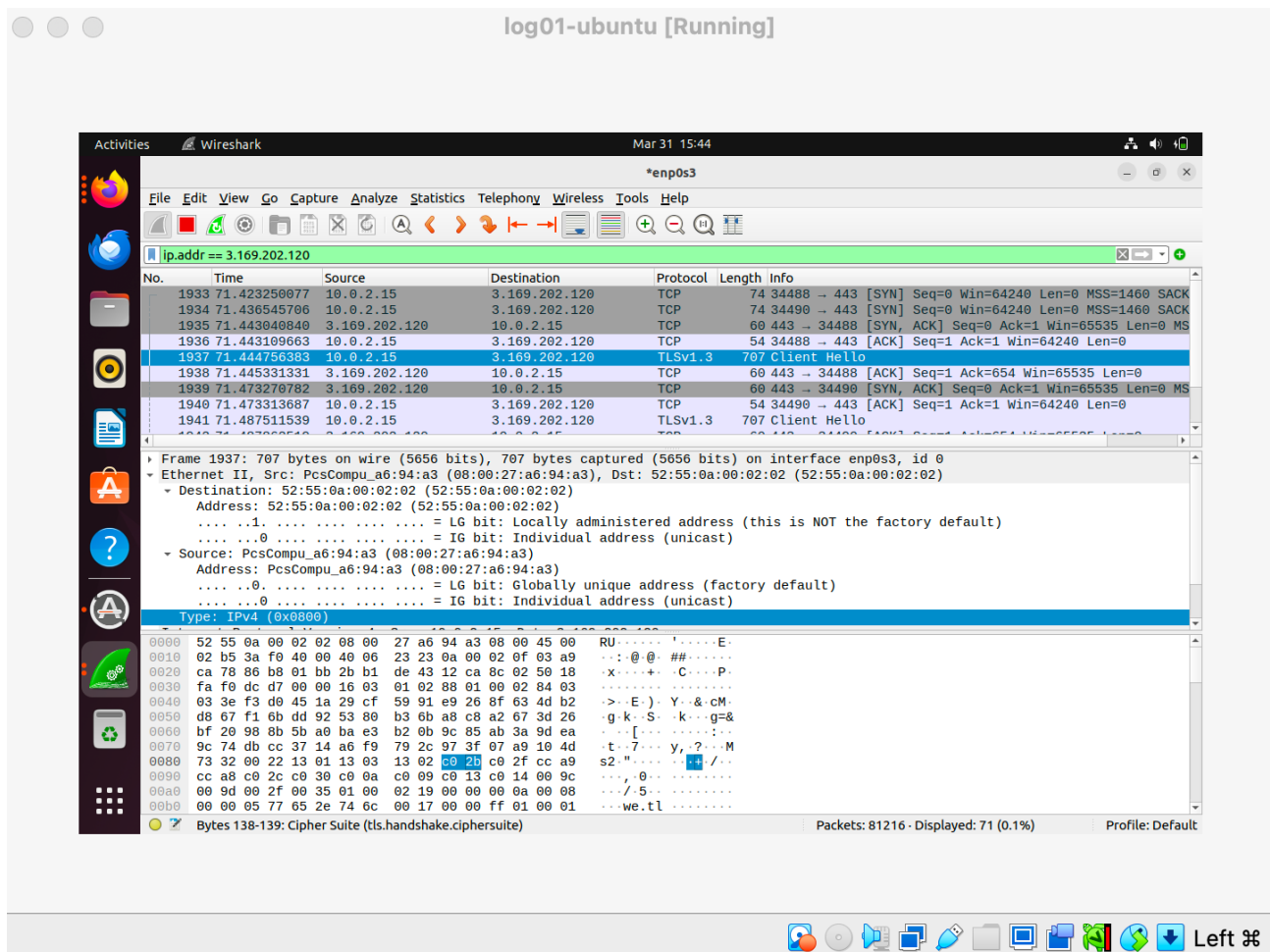


Figure 5. IP-based filtering used to investigate the full connection lifecycle associated with `we.tl`.

4. Connection Lifecycle Review

I reviewed the full lifecycle of the filtered session to understand whether the traffic pattern suggested ordinary web use or behavior more consistent with automated transfer. The sequence began with SYN, SYN-ACK, and ACK packets to establish the connection. It then transitioned into TLS negotiation and encrypted application data exchange. Later packets showed TCP keep-alive traffic, indicating that the session remained open, and FIN/ACK packets confirmed a clean close. These details allowed me to assess the session as controlled and structured rather than obviously erratic or malformed.

Security relevance: Lifecycle review helps analysts distinguish between random packet artifacts and meaningful communication patterns. This is especially valuable when the content itself is hidden by encryption.

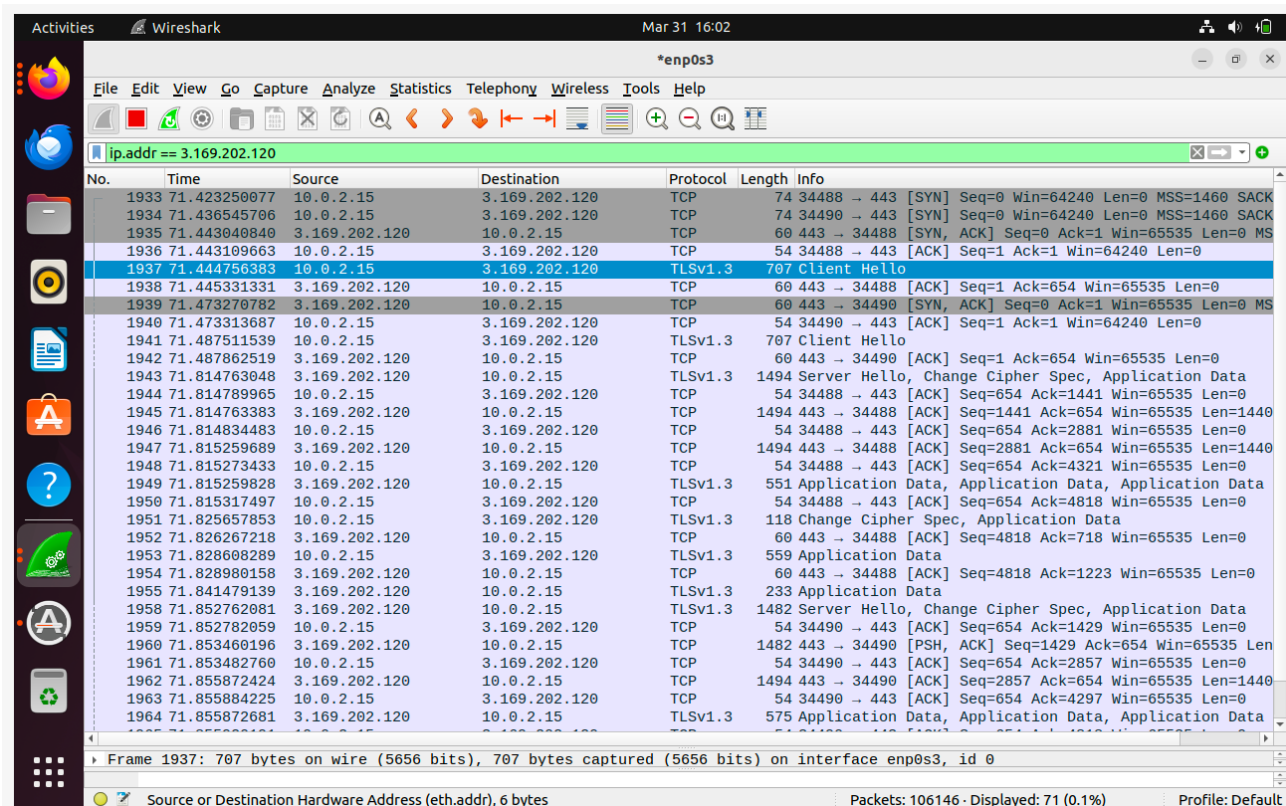


Figure 6. Session establishment and encrypted application-data exchange associated with the identified destination IP.

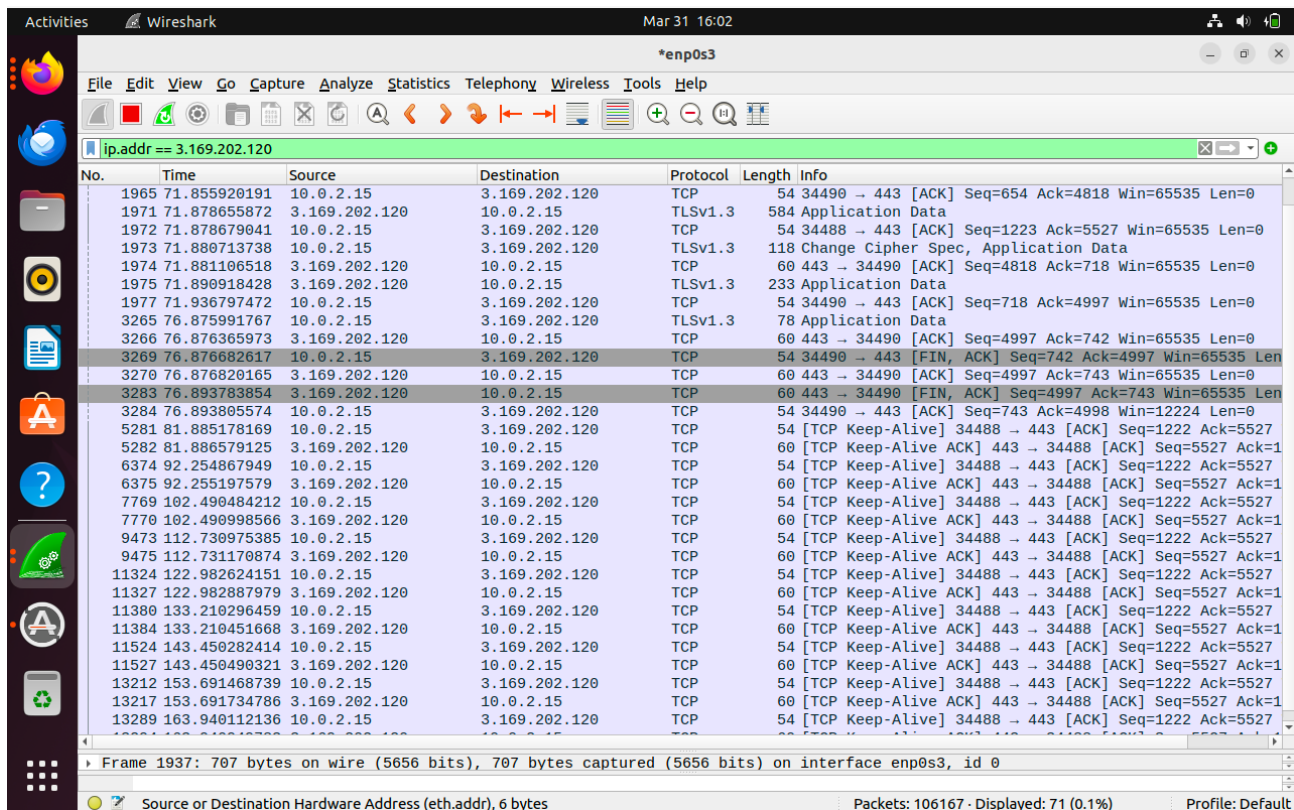


Figure 7. TCP keep-alive behavior showing persistent but orderly session maintenance.

No.	Time	Source	Destination	Protocol	Length	Info
3283	76.893783854	3.169.202.120	10.0.2.15	TCP	60	443 → 34490 [FIN, ACK] Seq=4997 Ack=743 Win=65535 Len=0
3284	76.893805574	10.0.2.15	3.169.202.120	TCP	54	34490 → 443 [ACK] Seq=743 Ack=4998 Win=12224 Len=0
5281	81.885178169	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
5282	81.886579125	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
6374	92.254867949	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
6375	92.255197579	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
7769	102.490484212	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
7770	102.490998566	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
9473	112.730975385	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
9475	112.731170874	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
11324	122.982624151	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
11327	122.982887979	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
11380	133.210296459	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
11384	133.210451668	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
11524	143.450282414	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
11527	143.450490321	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
13212	153.691468739	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
13217	153.691734786	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
13289	163.940112136	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
13294	163.940949783	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
13417	174.178449485	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
13420	174.180534140	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
14035	184.414827459	10.0.2.15	3.169.202.120	TCP	54	[TCP Keep-Alive] 34488 → 443 [ACK] Seq=1222 Ack=5527
14039	184.415500676	3.169.202.120	10.0.2.15	TCP	60	[TCP Keep-Alive ACK] 443 → 34488 [ACK] Seq=5527 Ack=1
14129	187.278933668	10.0.2.15	3.169.202.120	TLSv1.3	78	Application Data
14130	187.2800093780	3.169.202.120	10.0.2.15	TCP	60	443 → 34488 [ACK] Seq=5527 Ack=1247 Win=65535 Len=0
14131	187.280672743	10.0.2.15	3.169.202.120	TCP	54	34488 → 443 [FIN, ACK] Seq=1247 Ack=5527 Win=65535 Len=0
14132	187.281246682	3.169.202.120	10.0.2.15	TCP	60	443 → 34488 [ACK] Seq=5527 Ack=1248 Win=65535 Len=0
14133	187.300280424	3.169.202.120	10.0.2.15	TCP	60	443 → 34488 [FIN, ACK] Seq=5527 Ack=1248 Win=65535 Len=0
14134	187.300315076	10.0.2.15	3.169.202.120	TCP	54	34488 → 443 [ACK] Seq=1248 Ack=5528 Win=15104 Len=0

Frame 1937: 707 bytes on wire (5656 bits), 707 bytes captured (5656 bits) on interface enp0s3, id 0

Source or Destination Hardware Address (eth.addr), 6 bytes Packets: 106210 · Displayed: 71 (0.1%) Profile: Default

Figure 8. Clean connection termination reflected in FIN/ACK session closure.

Challenges and Troubleshooting

One of the main challenges in this project involved interpreting traffic in an environment where encryption limited direct visibility. At first, I expected HTTP-style content inspection to reveal the external domain directly, but that assumption was incorrect because we.tl communication appeared primarily as encrypted TLS traffic. Solving that problem required adjusting the analysis approach: first using TLS SNI to detect the domain, then pivoting to IP-based filtering to investigate the broader session. This troubleshooting process reflected a real-world reality of cybersecurity work: strong analysis often depends on changing methods when the first approach does not answer the actual question.

Evidence Summary

- successful packet capture on the Ubuntu NAT-facing interface
- baseline HTTP visibility into expected Firefox and browser-related traffic
- detection of outbound encrypted communication to we.tl via TLS SNI
- extraction and investigation of the associated destination IP address
- full-session visibility into handshake, application data, keep-alive behavior, and termination
- context-based interpretation of technically normal but operationally sensitive traffic

What I Learned

This project strengthened my practical understanding of how to solve security problems in environments where visibility is incomplete. I learned that effective traffic analysis is not just about recognizing packets, but about establishing normal behavior, identifying contextually meaningful outliers, and adapting methods when encryption limits direct inspection. I also gained more confidence in connecting packet-level observations to real security questions such as external data transfer risk, behavioral baselining, and the difference between technically normal traffic and activity that still warrants analyst attention.

Why This Project Matters

This project reflects skills that are directly relevant to entry-level cybersecurity, SOC analysis, incident response, and security operations. It demonstrates hands-on experience with packet capture, HTTP and TLS analysis, connection-lifecycle review, metadata-based threat evaluation, and technical documentation. More importantly, it positions me as a problem solver by showing that I can move from raw evidence to practical judgment. This is an important capability in environments where analysts must explain not only what they observed, but whether it matters and why.

Next Steps

- correlate network findings with alerting data in Stellar Cyber for stronger validation
- compare browser-based file-sharing traffic against other legitimate cloud-service baselines
- introduce lightweight detection logic for context-sensitive outbound destinations
- map observed behavior to a MITRE ATT&CK-informed narrative such as exfiltration over web services